

金仓数据库适配：AI 辅助研发经验分享

项目：ThingsBoard 4.1 适配人大金仓 KingbaseES V8 (V008R006C009B0014) 周期：2026-08-27 ~ 2026-09-21
(日历跨度近 4 周，实际投入约 **2 周 / 10 人日**) 交付：分支 `feature/kingbase-V008R006C009B0014-pg-oracle` |
工具：Claude Code | 面向：研发中心内部

一、一页总览

指标	纯人工（评估）	本次（人 + AI）
工作量	32 人日	10 人日（效率约 3 倍）
业务代码改动	预计需改查询 + 方言	0 行 （只加 1 个 SQL 函数 + 安装期自动处理）
对原 PostgreSQL 的影响	难以证明	实测为零 （同库对照，失败清单完全一致）
验证覆盖	抽测关键场景	950 用例 × 4 环境 × 多轮

三个核心结论：

- AI 把"做方案、跑验证"的成本降到很低，所以可以先把方案做出来，再判断要不要。
- 人的价值转移到了"定边界、砍范围、做判断"上，本次最关键的一步就是人叫停了 AI 已经跑通的大改方案。
- 一套方法可以直接复制：真实环境验证 → 基线对照 → 矩阵回归 → 人做取舍 → 干净交付。

二、五个亮点

亮点 1：零业务代码改动完成国产库适配

- 怎么做到的**：没有直接改代码，而是让 AI 在多环境全量跑测试，把 oracle 模式的 2740 条报错归结到**唯一根因**：金仓只自带两个参数的 `concat`。最后补一个 3 行 SQL 转发函数就解决了。
- 价值**：交付从"改 165 处查询 + 方言层"缩到"0 行 Java 改动"。上线风险、评审成本、后续升级合并成本都降到最低。

亮点 2：用"基线对照"证明零影响，也排掉了假问题

- 怎么做到的**：先用**未修改的原始代码**在 PostgreSQL 和金仓上各跑一轮作为基线，之后只看"基线没有、现在有"的失败。
- 价值**：一是找出了 10 个和金仓无关的老问题，避免为假问题做适配；二是"对 PG 零影响"有了数据证明，而不是口头保证。

亮点 3：先做后砍，敢于推翻已跑通的方案

- 怎么做到的**：三模式方案 AI 用约 10 小时就完整做了出来（改 165 处查询 + 方言层），功能是通的。我评估风险后叫停，一期只做 pg + oracle，并从 master 重开干净分支。
- 价值**：过去完整做一版方案要几周，没人敢推翻；现在试错成本低到可以"做出来再决定"。**决策依据是实测结果，而不是拍脑袋。**

亮点 4：AI 上机全流程闭环

- **怎么做到的**：把测试机凭据交给 Claude Code，它自己 SSH 登录、装数据库、装授权、切兼容模式、跑测试、看日志、排障。一开始给的库连不通，它自己报告并换了另一台继续。
- **价值**：原来 6 天的环境准备和熟悉工作大幅压缩，人不用在中间转发每一步命令和日志。

亮点 5：安装期"防呆"设计，问题不留到现场

- **怎么做到的**：安装时自动执行"探测模式 → 幂等检查 → 权限预检 → 执行补丁 → 结果复验"五步，任何一步不通过都中断安装，并打印修复指引。
- **价值**：实施人员不需要懂金仓细节，也不会出现"装成功了，跑起来才报错"。

三、十个可复用的技巧

#	技巧	一句话口诀	本次怎么用的	提示词示例
1	边界先行	先说不能动什么，再说要做什么	第一句就要求"不影响原有功能"，所有方案都按这条衡量	"可以开发，但必须保证不影响原有功能"
2	真机实测	不让 AI 凭知识回答	金仓很多行为文档里查不到，全部上机实测得出	"机器凭据给你，你直接验证给我结果"
3	基线对照	先跑原始代码，只看差异	识别出 10 个老问题；证明对 PG 零影响	"先用没改过的代码跑基线，只给我差异项"
4	矩阵回归	每次改动都跑全部环境	PG / 金仓 pg / 金仓 oracle 每次全量回归	"每次改完在所有环境都跑一遍"
5	后台长跑	长任务不盯屏	单轮 17 分钟、四环境一个多小时，挂后台跑通宵，早上看结论	"后台跑，跑完告诉我"
6	质疑改动量	AI 不会主动帮你砍需求	一句"去掉 mysql 是不是就不用改代码"，改动量从 165 处降到 0	"改动多吗？有风险吗？去掉 X 是不是就不用改了？"
7	要方案对比	让 AI 推荐，人来拍板	驱动、方言、补丁执行方式都让 AI 列对比再选	"给我两到三个方案的对比和代价，你更推荐哪个"
8	追问到懂	讲不出来就继续问	方言选型依据、连接串分离含义都是追问后才补全文档	"这个地方我没看懂"；"你是实测后才这么决定的吗"
9	干净重开	方案定型后从主干重开分支	重开两次，最终提交只有 7 个文件	"基于 master 新建分支，只合有用的代码，验证脚本不要"
10	结论沉淀	踩过的坑写下来，跨会话复用	靠 CLAUDE.md 和记忆文件记下版本号，C009 出现 448 个失败时没有误判成代码退化	"把这个结论记下来，以后换版本先重跑回归"

四、人必须把住的四道关

AI 能干活，但下面四类问题它不会替你兜底。共同特点是**都不报错**，测试可能还是绿的。

关口	本次的真实案例	怎么把住
范围与风险	三模式方案改了 165 处查询，AI 没有建议砍掉 mysql 模式	人来判断改动量和收益是否匹配
静默失效	AI 最初按 JDBC 产品名判断是不是金仓，但 PG 驱动返回的是 "PostgreSQL"，金仓逻辑全部没生效，review 才发现	测试全绿时多问一句"它到底走没走那条路径"
结论时效	C007 上验证通过的结论，在 C009 上完全失效	换小版本必须重跑回归
业务语义	mysql 模式强制大小写不敏感，CaseDev-A 和 casedev-a 被判为重复	业务决策交给业务侧确认

五、如果换一个人来做

工作量评估 32 人日：安装环境 2 天、申请授权 1 天、初始化三种模式 2 天、熟悉金仓 1 天、摸底 + 开发 20 天、自测修复 6 天。

最可能踩的坑（本次都真实遇到过）：

坑	后果	对应技巧
环境装好后不知道从哪下手	逐个报错去查，开发量主要耗在这里	基线对照、矩阵回归
直接让 AI"去适配"	改动面失控（本次一度改到 165 处查询）	边界先行、质疑改动量
用金仓自带驱动	需要配套修改方言，工作量大	要方案对比、真机实测
版本没对齐到小版本	C007 和 C009 模式都不同，结论全部失效	结论沉淀
把老问题当成金仓问题	为 10 个假问题做适配	基线对照
空串变 NULL、 变逻辑 OR	不报错，问题留到现场	真机实测、追问到懂

六、沉淀下来的团队资产

资产	内容	怎么复用
适配方法 SOP	确认精确版本 → AI 上机装环境 → 定边界 → 基线对照摸底 → 人做取舍 → 开发 + 矩阵回归 → 干净分支交付 → 真实环境实跑	达梦、海量等其他国产库，以及中间件适配直接套用
提示词模板	评估、归因、决策、收尾四个阶段的模板（见附录）	团队成员改环境信息即可使用
测试数据源参数化	同一套 dao 测试可以通过环境变量指向任意外部数据库	以后任何 PG 兼容库都能直接跑全量回归
一键部署脚本	docker/kingbase/docker-compose.yml，自动校验模式、建库、打补丁	测试和交付环境快速搭建
踩坑清单	版本、驱动、空串、concat、mysql 模式限制等	写入部署文档 docs/kingbase-pg-oracle.md
AI 上下文	CLAUDE.md + 记忆文件	新会话自动带入项目背景和实测结论

推广建议：

- 1. 把第三节的十个技巧和附录的提示词模板作为团队 AI 使用规范推广。
- 2. 下一个国产化适配任务按第六节的 SOP 执行，验证方法的可复制性。
- 3. 金仓 mysql 模式如果确有需求，二期单独立项，先由业务侧确认大小写不敏感是否可以接受。

附录 A：过程回顾

阶段	时间	做了什么	关键指令（原话）
1. 可行性评估	8/27 - 9/02	AI 登录测试机、装授权、跑探针 SQL，出第一版实现（12 个提交）	"机器登录密码给你，你直接验证给我结果"
2. 文档化 + 追问	9/14 - 9/17	AI 基于代码反向生成设计文档，看不懂的逐条追问	"使用金仓的方言改动大对吗？"
3. 基线对照	9/18	原始代码在金仓 pg 模式跑测试，与 PG 逐项一致，pg 模式零适配	"直接用原来没有修改的任何代码跑"
4. 三模式后叫停	9/20	AI 实现三模式（165 处查询 + 方言层），评估风险后叫停	"现在三种模式改动代码太多了，会不会存在风险"
5. 零改动交付	9/20 - 9/21	从 master 重开分支，补 1 个 SQL 函数 + 安装期自动处理，矩阵验证，71 环境实跑一次通过	"安装期自动执行，没有权限就报错"

最终验证矩阵（950 个用例，PostgreSQL 15.4 为基线）：

环境	代码	Failures	Errors	与基线对比
PostgreSQL 15.4	原始代码	4	6	基线
金仓 pg 模式	未改动	4	6	逐项一致
金仓 oracle 模式	未改动、无补丁	128	320	+441
金仓 oracle 模式	未改动、有补丁	4	6	逐项一致

附录 B：提示词模板

评估阶段

帮我评估当前项目切换到 <目标环境> 是否可行。<环境地址与凭据>，你直接登录验证给我结果，不要只做静态分析。前提是不能影响现有 <原有环境> 的任何功能，不可用的功能单独整理给我。

归因阶段

先用未修改的原始代码在 <目标环境> 跑一轮完整测试作为基线，再用改动后的代码跑一轮，只把"基线没有、现在有"的失败列给我。后台跑，跑完告诉我。

决策阶段

这个方案改动多吗？会不会有风险？如果不考虑 <某个范围>，是不是代码就不用动了？给我两到三个方案的对比，说明各自代价，并说明你更推荐哪个。

收尾阶段

基于最新的 master 新建分支 <分支名>，只把有用的代码合过去，验证脚本和中间产物不要。然后 review 本次改动：改了哪些文件、为什么改、对原有路径有什么影响。最后基于实际代码生成设计文档和部署手册，放到 docs 目录。